Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology
Department of Computer Science & Engineering

LAB MANUAL

Course No: CSE 4120

Course Name: Data Mining Sessional

Semester: 4th Year Odd Semester

Prepared By:

Dr. Julia Rahman
Associate Professor
Department of CSE, RUET

A. F. M. Minhazur Rahman
Assistant Professor
Department of CSE, RUET

Rajshahi, Bangladesh

Data Mining Laboratory Guidelines

Before Coming to Lab

- Read the lab sheet/manual carefully before class.
- Review relevant theory: preprocessing, statistics, and Python syntax.
- Ensure Python 3 and required libraries (pandas, numpy, sklearn, seaborn, matplotlib) are installed.
- Prepare short answers or pseudocode if assigned.

During Lab Session

A. Conduct

- Be punctual and maintain professional etiquette.
- Keep your code modular and properly commented.

B. Work Process

- Perform each experiment individually or in assigned groups.
- Validate outputs and record results in notebook or Jupyter.
- Interact with instructors for clarification.

After Completing Experiment

- Demonstrate your implementation for evaluation.
- Submit a report including Objective, Theory, Code, Outputs, Discussion, and Conclusion.
- Clean up your workstation and upload files if required.

Evaluation Criteria

Criteria	Description
Preparation	Pre-lab readiness, environment setup
Implementation	Correct use of Python libraries and workflow
Understanding	Ability to explain data handling steps and logic
Reporting	Completeness and clarity of report with plots
Discipline	Attendance, punctuality, lab behavior

CO–PO Mapping for CSE 4120

CO	CO Statement	Mapped PO	Assessment Tools
CO1	Understand fundamental data-mining concepts and preprocessing methods.	PO1	Lab Performance, Report, Viva
CO2	Apply data-mining techniques to discover patterns and correlations.	PO2	Lab Performance, Report, Viva
CO3	Design classification and clustering models for predictive analysis.	PO3	Lab Performance, Report, Viva

Course Overview

Course Code: CSE 4120
Credit: 0.75
Semester: 4th Year Odd

Course Rationale

Data mining plays a vital role in extracting useful knowledge from large datasets for decision-making in business, healthcare, and engineering. This sessional provides hands-on training in data preprocessing, pattern mining, classification, clustering, and predictive analysis using modern tools and libraries.

Course Objectives

- Understand fundamental data-mining concepts, preprocessing and evaluation methods.
- Apply algorithms to discover patterns and associations in real datasets.
- Design and implement classification and clustering models.
- Evaluate results using appropriate metrics and visualizations.
- Use modern Python-based tools for data analysis.

Course Content

Introduction to Data Mining; Data Preprocessing; Mining Frequent Patterns and Associations; Classification; Clustering and Prediction; Advanced Analyses.

Experiment Plan

Week	Experiment Topic	Delivery Method	Aligned CO
1	Introduction to Data Mining and Tools	Lecture, Lab Manual 1	CO1
2	Data Preprocessing	Lecture, Lab Manual 2	CO1
3	Exploratory Data Analysis (EDA)	Lecture, Lab Manual 3	CO2
4	Mining Frequent Itemsets and Association Rules	Lecture, Lab Manual 4	CO2
5	Classification Techniques	Lecture, Lab Manual 5	CO3
6	Clustering (k-Means, Hierarchical)	Lecture, Lab Manual 6	CO3
7	Viva and Project Evaluation	Discussion, Presentation	All

Learning Materials

1. Jiawei Han, Micheline Kamber, Jian Pei, *Data Mining: Concepts and Techniques*.
2. Ian H. Witten et al., *Data Mining: Practical Machine Learning Tools and Techniques*.
3. Scikit-learn and Pandas official documentation.

Contents

Lab 1: Environment Setup and Data Loading

Objective

To set up the Python data science environment using Miniconda, install essential libraries (pandas, numpy, scikit-learn, matplotlib, seaborn), and explore initial data loading and visualization commands.

Theory

Data mining experiments rely on reproducible environments and a consistent software stack. Python, with its scientific libraries, offers a flexible and widely supported ecosystem. Conda environments allow isolated package management, avoiding version conflicts.

Tools Required

- Miniconda or Anaconda
- Python 3.10 or later
- JupyterLab / VS Code
- Libraries: pandas, numpy, scikit-learn, seaborn, matplotlib

Installation Procedure

A. Windows Installation

1. Download **Miniconda** (Windows 64-bit) from `https://docs.conda.io/en/latest/miniconda.html`
2. Run the installer → "Add Miniconda to PATH".
3. Open **Anaconda Prompt** and execute:

```
conda create -n dm-lab python=3.11 -y
conda activate dm-lab
conda install -c conda-forge pandas numpy scikit-learn
↪ matplotlib seaborn jupyterlab -y
```

4. Verify installation:

```
python -c "import pandas, numpy, sklearn, matplotlib, seaborn;
↪ print('All OK!')"
```

B. Linux / macOS Installation

1. Download installer:

```
wget
↪ https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh
source ~/.bashrc
```

2. Create and activate environment:

```
conda create -n dm-lab python=3.11 -y
conda activate dm-lab
```

3. Install required packages (same as above).

Sample Code: Loading and Inspecting Data

Python Code

```
# lab1_environment.py
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Load dataset
df = pd.read_csv("data/sample.csv")  # replace with actual path

# 2. Inspect top rows
print("Head:\n", df.head())

# 3. Overview of dataset
print("\nInfo:")
print(df.info())

# 4. Statistical summary
print("\nDescribe:")
print(df.describe())

# 5. Missing values
print("\nMissing values per column:")
print(df.isnull().sum())

# 6. Visualization
sns.pairplot(df.select_dtypes(include=np.number))
plt.show()
```

Explanation of Key Functions

- `pd.read_csv()`: reads CSV data into DataFrame.
- `df.head(x)`: displays top x records.
- `df.info()`: shows column types and non-null counts.
- `df.describe()`: returns statistical summary.
- `df.isnull().sum()`: checks missing values.
- `sns.pairplot()`: visualizes pairwise relationships between numerical features.

Discussion

Proper environment setup ensures reproducibility and minimizes dependency issues. Using pandas and seaborn, students can quickly perform exploratory data analysis (EDA), which is foundational to all later labs on preprocessing, pattern mining, and model building.

Exercises

1. Create the conda environment `dm-lab` and install required packages.
2. Load a public dataset (e.g., *Iris or Titanic*) and display its `head()`, `info()`, and `describe()`.
3. Identify columns with missing values and report them.
4. Produce at least one visualization using matplotlib or seaborn and interpret it.

References

- https://docs.conda.io/
- https://pandas.pydata.org/docs/
- https://seaborn.pydata.org/

Lab 2: Data Preprocessing and Cleaning

Objective

To understand and implement basic data preprocessing techniques such as handling missing values, encoding categorical features, normalization, and feature scaling using `pandas` and `scikit-learn`.

Theory

Raw data collected from real-world sources often contains noise, missing entries, or inconsistent formats. Data preprocessing aims to clean and transform such data into a structured form suitable for model training. Common preprocessing tasks include:

- Handling missing or null values.
- Encoding categorical data into numeric form.
- Normalizing or scaling numerical attributes.
- Removing duplicate or irrelevant records.

Major Functions Used

- `df.isnull().sum()`: Counts missing values per column.
- `df.fillna(value)`: Fills missing values with a constant, mean, or mode.
- `df.dropna()`: Removes rows or columns with missing data.
- `pd.get_dummies()`: Performs one-hot encoding for categorical variables.
- `StandardScaler()`, `MinMaxScaler()`: From `sklearn.preprocessing`, used for normalization and scaling.

Procedure

1. Load Data and Inspect

```python
import pandas as pd
import numpy as np

df = pd.read_csv("data/dirty_data.csv")

print("Initial shape:", df.shape)
print("\nMissing values:\n", df.isnull().sum())
print("\nData types:\n", df.dtypes)
```

2. Handle Missing Values

```python
# Option 1: Drop rows with any missing value
df_drop = df.dropna()

# Option 2: Fill missing values
df_fill = df.fillna({
    'Age': df['Age'].mean(),
    'Salary': df['Salary'].median(),
    'City': 'Unknown'
})
```

Explanation: Dropping removes incomplete data but may lead to information loss. Filling preserves data count and is preferred when missing proportion is small.

3. Encode Categorical Data

```python
# One-hot encoding using pandas
df_encoded = pd.get_dummies(df_fill, columns=['City', 'Gender'],
    drop_first=True)

print(df_encoded.head())
```

Explanation: Converts categorical variables like "Male/Female" or "Dhaka/Rajshahi" into binary columns, allowing numerical algorithms to process them.

4. Normalize and Scale Numeric Columns

```python
from sklearn.preprocessing import StandardScaler, MinMaxScaler

num_cols = ['Age', 'Salary']

# Standardization: mean = 0, std = 1
scaler_std = StandardScaler()
df_std = df_encoded.copy()
df_std[num_cols] = scaler_std.fit_transform(df_std[num_cols])

# Normalization: range [0, 1]
scaler_minmax = MinMaxScaler()
df_norm = df_encoded.copy()
df_norm[num_cols] = scaler_minmax.fit_transform(df_norm[num_cols])
```

Explanation: Scaling ensures that all features contribute equally to the model and prevents bias toward larger numeric values.

5. Remove Duplicates

```
before = df_norm.shape[0]
df_clean = df_norm.drop_duplicates()
after = df_clean.shape[0]
print(f"Removed {before - after} duplicate rows")
```

6. Full Example Script

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

# Load data
df = pd.read_csv("data/dirty_data.csv")

# Fill missing values
df.fillna({
    'Age': df['Age'].mean(),
    'Salary': df['Salary'].median(),
    'City': 'Unknown'
}, inplace=True)

# Encode categorical columns
df = pd.get_dummies(df, columns=['City', 'Gender'], drop_first=True)

# Standardize numeric columns
scaler = StandardScaler()
num_cols = ['Age', 'Salary']
df[num_cols] = scaler.fit_transform(df[num_cols])

# Drop duplicates
df.drop_duplicates(inplace=True)

print(df.head())
```

Listing 1: Complete Preprocessing Example

Discussion

Proper data preprocessing is essential before applying any data mining or machine learning algorithm. Inconsistent or unscaled data leads to incorrect model training, while imputation and normalization improve accuracy and convergence.

Exercises

1. Load a dataset with missing and categorical values (e.g., Titanic dataset).
2. Try three imputation methods: mean, median, and mode. Compare their effects.
3. Apply one-hot encoding and observe the change in column count.
4. Normalize numeric columns using both StandardScaler and MinMaxScaler.
5. Write a short report explaining how preprocessing impacted your dataset.

Outcome

Students will gain hands-on experience cleaning datasets, applying imputation, encoding, and scaling—crucial preprocessing steps for all subsequent labs.

Lab 3: Exploratory Data Analysis (EDA)

Objective

To perform exploratory data analysis (EDA) on a dataset using Python libraries such as `pandas`, `matplotlib`, and `seaborn`, and to interpret the key patterns, trends, and relationships between variables.

Theory

Exploratory Data Analysis is the process of visually and quantitatively summarizing data to identify patterns, anomalies, and insights before model building. It provides an intuitive understanding of the dataset, highlighting distributions, correlations, and possible outliers.

Common EDA Tasks:

- Summarizing data using descriptive statistics.
- Visualizing feature distributions and relationships.
- Detecting correlations and multicollinearity.
- Identifying missing or abnormal data patterns.

Major Functions Used

- `df.describe()` – Basic statistical summary.
- `df.corr()` – Pairwise correlation matrix.
- `sns.heatmap()` – Visual correlation map.
- `sns.histplot()`, `sns.boxplot()`, `sns.pairplot()` – Univariate and multivariate visualization.

Procedure

1. Import Libraries and Load Data

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load cleaned dataset from previous lab
df = pd.read_csv("data/cleaned_data.csv")

print("Shape:", df.shape)
print(df.head())
```

2. Basic Statistical Summary

```python
# Descriptive statistics for numerical features
print(df.describe())

# Information about dataset
print(df.info())

# Unique values per column
print(df.nunique())
```

Explanation: This step provides mean, median, standard deviation, and count, helping you quickly identify feature ranges and missing or abnormal values.

3. Visualize Distributions

```python
# Histogram and density plot
sns.histplot(df['Age'], kde=True)
plt.title("Distribution of Age")
plt.show()

# Boxplot for salary to detect outliers
sns.boxplot(x=df['Salary'])
plt.title("Salary Distribution and Outliers")
plt.show()
```

Observation: Histograms reveal shape (normal, skewed, bimodal). Boxplots highlight median, quartiles, and extreme values (outliers).

4. Relationship Between Variables

```python
# Pairwise relationships among numeric features
sns.pairplot(df.select_dtypes('number'))
plt.suptitle("Pairwise Relationships", y=1.02)
plt.show()
```

Explanation: Pairplots show relationships and correlations between all numeric columns. Clusters or linear patterns often indicate potential predictive relationships.

5. Correlation Heatmap

```
corr_matrix = df.corr(numeric_only=True)
plt.figure(figsize=(8,6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm',
   linewidths=0.5)
plt.title("Correlation Heatmap")
plt.show()
```

Observation: Correlation coefficients close to +1 or -1 show strong linear relationships. High correlation among features may cause multicollinearity in models.

6. Categorical Variable Analysis

```
# Count plot for categorical features
sns.countplot(x='Gender', data=df)
plt.title("Gender Distribution")
plt.show()

# Cross-tabulation: Gender vs City
pd.crosstab(df['Gender'], df['City'], normalize='index') * 100
```

Explanation: Count plots show category frequencies. Cross-tabulation (or contingency tables) helps understand interaction between two categorical variables.

7. Outlier Detection (Z-Score Method)

```
from scipy import stats

z = np.abs(stats.zscore(df.select_dtypes('number')))
outliers = (z > 3).sum(axis=0)
print("Outliers per column:\n", outliers)
```

Explanation: Z-score method detects extreme values beyond three standard deviations, helping decide if they should be removed or adjusted.

8. Full EDA Script

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("data/cleaned_data.csv")

# Summary statistics
print(df.describe())

# Correlation heatmap
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap='coolwarm')
```

```
plt.show()

# Histograms and pairplots
sns.histplot(df['Age'], kde=True)
plt.show()

sns.pairplot(df.select_dtypes('number'))
plt.show()
```

Listing 2: Complete EDA Workflow

Discussion

EDA reveals the structure and distribution of data, outliers, and relationships between features. Understanding these aspects is critical for selecting appropriate preprocessing and modeling techniques. Visual analysis often exposes issues invisible in numerical summaries.

Exercises

1. Load a dataset (e.g., Titanic, Iris, or any Kaggle dataset).
2. Compute descriptive statistics and interpret results.
3. Create histograms, boxplots, and heatmaps for key variables.
4. Identify two strong and two weak correlations.
5. Write short interpretations of the discovered relationships.

Outcome

Students will be able to perform descriptive and visual analyses, recognize patterns and outliers, and draw meaningful inferences from raw datasets — preparing them for feature engineering and model development in later labs.

Lab 4: Mining Frequent Patterns and Association Rules

Objective

To discover frequent itemsets and generate association rules from transactional data using the `Apriori` and `FP-Growth` algorithms implemented in the `mlxtend` library.

Theory

Association rule mining finds interesting relationships (associations, correlations) between variables in large datasets. It is a fundamental data mining technique used in market basket analysis, recommendation systems, and behavior analytics.

Definitions:

- **Itemset:** A collection of one or more items.
- **Support:** Fraction of transactions containing an itemset.
- **Confidence:** Probability that a rule holds, given the antecedent.
- **Lift:** Measure of how much more often items occur together than expected if they were independent.

Algorithms:

- **Apriori:** Iteratively builds larger frequent itemsets using previously found frequent subsets.
- **FP-Growth:** Constructs an FP-tree to mine itemsets without candidate generation, faster for dense datasets.

Major Functions Used

- `mlxtend.frequent_patterns.apriori()`: Generates frequent itemsets.
- `mlxtend.frequent_patterns.fpmax() / fpgrowth()`: Alternative faster algorithms.
- `mlxtend.frequent_patterns.association_rules()`: Derives rules from frequent itemsets.

Procedure

1. Install and Import Packages

```
conda install -c conda-forge mlxtend -y
# or
pip install mlxtend
```

```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules,
    fpgrowth
```

2. Load and Prepare Transaction Data

We will create a small dataset representing a supermarket transaction list.

```
dataset = [
    ['Milk', 'Bread', 'Butter'],
    ['Bread', 'Eggs', 'Butter', 'Jam'],
    ['Milk', 'Bread', 'Eggs'],
    ['Milk', 'Bread', 'Butter', 'Eggs'],
    ['Bread', 'Butter']
]

# Convert dataset to one-hot encoded DataFrame
from mlxtend.preprocessing import TransactionEncoder

te = TransactionEncoder()
te_ary = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_ary, columns=te.columns_)

print(df.head())
```

Explanation: Each transaction is transformed into a Boolean matrix indicating the presence (True/False) of each item.

3. Apply Apriori Algorithm

```
# Find frequent itemsets with minimum support = 0.4
frequent_itemsets = apriori(df, min_support=0.4, use_colnames=True)
print(frequent_itemsets)
```

Output:

```
   support           itemsets
0      0.8            (Bread)
1      0.6            (Butter)
2      0.6              (Milk)
3      0.6              (Eggs)
4      0.4    (Bread, Butter)
...
```

4. Generate Association Rules

```
rules = association_rules(frequent_itemsets, metric="confidence",
   min_threshold=0.6)
print(rules[['antecedents','consequents','support',
'confidence','lift']])
```

Sample Output:

```
  antecedents     consequents  support  confidence  lift
0    (Bread)          (Butter)    0.4       0.67      1.11
1    (Butter)         (Bread)     0.4       0.67      1.11
2    (Milk, Bread)    (Eggs)      0.4       0.75      1.25
```

Interpretation: The rule *(Bread → Butter)* means that 67% of customers who bought Bread also bought Butter, and the lift > 1 indicates a positive association.

5. Apply FP-Growth Algorithm

```
frequent_fp = fpgrowth(df, min_support=0.4, use_colnames=True)
rules_fp = association_rules(frequent_fp, metric="lift",
   min_threshold=1.0)
print(rules_fp[['antecedents','consequents','support',
'confidence','lift']])
```

Explanation: FP-Growth avoids generating all candidate itemsets like Apriori, making it faster for large transactional datasets.

6. Visualization of Frequent Itemsets

```
import matplotlib.pyplot as plt

frequent_itemsets.sort_values('support', ascending=False,
   inplace=True)
plt.bar(x=range(len(frequent_itemsets)),
   height=frequent_itemsets['support'])
plt.xticks(range(len(frequent_itemsets)),
           [', '.join(x) for x in frequent_itemsets['itemsets']],
   rotation=45)
plt.title("Frequent Itemsets by Support")
plt.ylabel("Support")
plt.show()
```

Observation: Bar plots of support values help quickly identify the most common item combinations.

7. Full Example Script

```python
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules, fpgrowth

dataset = [
    ['Milk', 'Bread', 'Butter'],
    ['Bread', 'Eggs', 'Butter', 'Jam'],
    ['Milk', 'Bread', 'Eggs'],
    ['Milk', 'Bread', 'Butter', 'Eggs'],
    ['Bread', 'Butter']
]

# Convert to one-hot DataFrame
te = TransactionEncoder()
te_ary = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_ary, columns=te.columns_)

# Apriori
freq_ap = apriori(df, min_support=0.4, use_colnames=True)
rules_ap = association_rules(freq_ap, metric="confidence", min_threshold=0.6)
print("Apriori Rules:\n", rules_ap[['antecedents','consequents','support','confidence','lift']])

# FP-Growth
freq_fp = fpgrowth(df, min_support=0.4, use_colnames=True)
rules_fp = association_rules(freq_fp, metric="lift", min_threshold=1.0)
print("\nFP-Growth Rules:\n", rules_fp[['antecedents','consequents','support','confidence','lift']])
```

Listing 3: Complete Apriori and FP-Growth Implementation

Discussion

Both Apriori and FP-Growth discover frequent co-occurrences of items, but FP-Growth is computationally more efficient. These algorithms are crucial in recommendation systems (e.g., "people who bought X also bought Y") and retail analytics.

Exercises

1. Create your own transactional dataset with at least 10 transactions.
2. Apply Apriori and FP-Growth to find itemsets with support ≥ 0.3.
3. Compare the number of rules generated by both algorithms.

4. Visualize top 10 itemsets using support values.
5. Interpret any two meaningful association rules.

Outcome

Students will be able to:

- Apply Apriori and FP-Growth algorithms using `mlxtend`.
- Interpret frequent itemsets, association rules, and evaluation metrics.
- Understand support, confidence, and lift in the context of market basket analysis.

Lab 5: Classification Techniques

Objective

To understand and implement different classification algorithms — Decision Tree, Naive Bayes, and Logistic Regression — using the `scikit-learn` library, and evaluate their performance with accuracy and confusion matrix.

Theory

Classification is a supervised learning task that assigns input data into predefined categories (classes) based on patterns learned from labeled training data.

Common Classification Algorithms

- **Decision Tree:** A tree-like model that splits data based on attribute values to maximize class purity (measured by Gini or Entropy).
- **Naive Bayes:** A probabilistic model based on Bayes' theorem, assuming feature independence.
- **Logistic Regression:** A linear model that predicts probabilities for categorical outcomes using the sigmoid function.

Evaluation Metrics:

- **Accuracy:** Fraction of correct predictions.
- **Confusion Matrix:** Table showing correct and incorrect classifications.
- **Precision, Recall, F1-Score:** Derived from the confusion matrix to measure class-wise performance.

Major Functions Used

- `train_test_split()`: Splits data into training and testing sets.
- `DecisionTreeClassifier`, `GaussianNB`, `LogisticRegression`: Classifier classes.
- `confusion_matrix()`, `classification_report()`: Performance evaluation.

Procedure

1. Import Required Libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
   classification_report
```

2. Load Dataset and Prepare Data

We'll use the popular **Iris dataset** available in `sklearn.datasets`.

```
from sklearn.datasets import load_iris

iris = load_iris()
X = iris.data
y = iris.target

# Split data into 80% training, 20% testing
X_train, X_test, y_train, y_test = train_test_split(X, y,
   test_size=0.2, random_state=42)
```

Explanation: Splitting ensures the model generalizes well and doesn't simply memorize training data.

3. Decision Tree Classifier

```
# Initialize and train model
dt = DecisionTreeClassifier(criterion='entropy', max_depth=3,
   random_state=0)
dt.fit(X_train, y_train)

# Predict and evaluate
y_pred_dt = dt.predict(X_test)
print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
print(confusion_matrix(y_test, y_pred_dt))
```

Observation: Decision Trees are easy to interpret but prone to overfitting if not pruned or limited in depth.

4. Naive Bayes Classifier

```
nb = GaussianNB()
nb.fit(X_train, y_train)

y_pred_nb = nb.predict(X_test)
```

```
print("Naive Bayes Accuracy:", accuracy_score(y_test, y_pred_nb))
print(confusion_matrix(y_test, y_pred_nb))
```

Observation: Naive Bayes works well for high-dimensional data and text classification, despite its strong independence assumptions.

5. Logistic Regression

```
lr = LogisticRegression(max_iter=200)
lr.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)
print("Logistic Regression Accuracy:", accuracy_score(y_test,
    y_pred_lr))
print(confusion_matrix(y_test, y_pred_lr))
```

Observation: Logistic Regression models linear decision boundaries and is effective for linearly separable classes.

6. Compare Classifier Performance

```
# Summary table
models = ['Decision Tree', 'Naive Bayes', 'Logistic Regression']
accuracies = [
    accuracy_score(y_test, y_pred_dt),
    accuracy_score(y_test, y_pred_nb),
    accuracy_score(y_test, y_pred_lr)
]

for m, acc in zip(models, accuracies):
    print(f"{m:20s} Accuracy: {acc*100:.2f}%")
```

Example Output:

```
Decision Tree         Accuracy: 96.67%
Naive Bayes           Accuracy: 96.67%
Logistic Regression   Accuracy: 100.00%
```

7. Classification Report and Visualization

```
import seaborn as sns
import matplotlib.pyplot as plt

# Choose one model (e.g., Decision Tree)
cm = confusion_matrix(y_test, y_pred_dt)
sns.heatmap(cm, annot=True, cmap='Blues', fmt='g')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix - Decision Tree')
plt.show()

```

```
print("Classification Report:\n")
print(classification_report(y_test, y_pred_dt,
    target_names=iris.target_names))
```

Explanation: A heatmap visually represents true vs predicted classes; the classification report gives detailed precision and recall.

8. Full Example Script

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
    classification_report

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data,
    iris.target,
    test_size=0.2, random_state=42)

models = {
    'Decision Tree': DecisionTreeClassifier(max_depth=3),
    'Naive Bayes': GaussianNB(),
    'Logistic Regression': LogisticRegression(max_iter=200)
}

for name, model in models.items():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    print(f"\n{name} Accuracy: {accuracy_score(y_test,
   preds)*100:.2f}%")
    print(confusion_matrix(y_test, preds))
```

Listing 4: Classification Techniques using Scikit-learn

Discussion

Each classification algorithm has strengths and weaknesses:

- Decision Trees provide interpretability but risk overfitting.
- Naive Bayes is fast and simple, ideal for high-dimensional data.
- Logistic Regression offers probabilistic outputs and works well for linear relationships.

Choosing the best model depends on the dataset and problem type.

Exercises

1. Apply these classifiers to a different dataset (e.g., Titanic or Wine dataset).
2. Vary the test size (e.g., 0.3, 0.4) and observe changes in accuracy.
3. Use `max_depth` and `criterion` parameters in Decision Tree and note the effect.
4. Compare accuracy, precision, recall, and F1-score among the three models.
5. Visualize the Decision Tree using `plot_tree()` from `sklearn.tree`.

Outcome

Students will:

- Learn to implement multiple classification models using `scikit-learn`.
- Understand how to evaluate and compare model performance.
- Develop the ability to select the most appropriate classification technique for a given dataset.

Lab 6: Clustering Techniques

Objective

To implement and compare unsupervised clustering algorithms — **K-Means** and **Hierarchical Clustering** — and evaluate their performance using visualization and silhouette analysis.

Theory

Clustering is an unsupervised learning task that groups data points into clusters based on similarity. Unlike classification, the data are unlabeled, and the goal is to discover inherent structure.

Common Clustering Algorithms

- **K-Means Clustering:** Partitions data into k clusters by minimizing within-cluster variance.
- **Hierarchical Clustering:** Builds a hierarchy of clusters through successive merging (agglomerative) or splitting (divisive).

Evaluation Metric:

- **Silhouette Score:** Measures how similar a point is to its own cluster compared to other clusters. Ranges from -1 (incorrect clustering) to $+1$ (well clustered).

Major Functions Used

- `KMeans(n_clusters, random_state)` — from `sklearn.cluster`
- `AgglomerativeClustering()` — for hierarchical clustering
- `silhouette_score()` — from `sklearn.metrics`
- `linkage()`, `dendrogram()` — from `scipy.cluster.hierarchy`

Procedure

1. Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import linkage, dendrogram
```

2. Generate Synthetic Data

```
# Create a 2D dataset with 3 clusters
X, y_true = make_blobs(n_samples=200, centers=3, cluster_std=0.70,
    random_state=42)
plt.scatter(X[:, 0], X[:, 1], s=50)
plt.title("Synthetic Dataset (3 Clusters)")
plt.show()
```

3. Apply K-Means Clustering

```
kmeans = KMeans(n_clusters=3, random_state=0)
labels_kmeans = kmeans.fit_predict(X)

# Plot results
plt.scatter(X[:, 0], X[:, 1], c=labels_kmeans, cmap='viridis', s=50)
plt.scatter(kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
            c='red', marker='X', s=200, label='Centroids')
plt.title("K-Means Clustering")
plt.legend()
plt.show()

print("Silhouette Score (K-Means):", silhouette_score(X,
    labels_kmeans))
```

Explanation: The K-Means algorithm iteratively updates centroids until cluster assignments stabilize. The silhouette score evaluates clustering quality — the higher, the better.

4. Apply Hierarchical Clustering

```
# Perform agglomerative hierarchical clustering
hc = AgglomerativeClustering(n_clusters=3, linkage='ward')
labels_hc = hc.fit_predict(X)

```

```
plt.scatter(X[:, 0], X[:, 1], c=labels_hc, cmap='plasma', s=50)
plt.title("Hierarchical Clustering (Agglomerative)")
plt.show()

print("Silhouette Score (Hierarchical):", silhouette_score(X,
   labels_hc))
```

Explanation: Agglomerative clustering starts with individual data points and merges them iteratively based on distance, forming a tree-like hierarchy.

5. Dendrogram Visualization

```
linked = linkage(X, method='ward')
plt.figure(figsize=(8, 5))
dendrogram(linked, truncate_mode='lastp', p=12, leaf_rotation=45)
plt.title("Dendrogram (Ward Linkage)")
plt.xlabel("Cluster Size")
plt.ylabel("Distance")
plt.show()
```

Observation: The dendrogram helps visualize merging distances and decide the optimal number of clusters.

6. Compare Both Methods

```
score_km = silhouette_score(X, labels_kmeans)
score_hc = silhouette_score(X, labels_hc)

print(f"K-Means Silhouette Score: {score_km:.3f}")
print(f"Hierarchical Silhouette Score: {score_hc:.3f}")

if score_km > score_hc:
    print("K-Means performs better on this dataset.")
else:
    print("Hierarchical clustering performs better.")
```

Interpretation: In general, K-Means performs better for spherical clusters, while hierarchical clustering may capture nested or irregular structures.

7. Full Example Script

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import linkage, dendrogram
import matplotlib.pyplot as plt

# Data generation
X, _ = make_blobs(n_samples=200, centers=3, random_state=42)

```

```
# K-Means
kmeans = KMeans(n_clusters=3, random_state=42)
labels_kmeans = kmeans.fit_predict(X)

# Hierarchical
hc = AgglomerativeClustering(n_clusters=3)
labels_hc = hc.fit_predict(X)

# Evaluation
print("Silhouette (K-Means):", silhouette_score(X, labels_kmeans))
print("Silhouette (Hierarchical):", silhouette_score(X, labels_hc))

# Visualization
plt.subplot(1,2,1)
plt.scatter(X[:,0], X[:,1], c=labels_kmeans, cmap='viridis')
plt.title("K-Means Clusters")

plt.subplot(1,2,2)
plt.scatter(X[:,0], X[:,1], c=labels_hc, cmap='plasma')
plt.title("Hierarchical Clusters")
plt.show()
```

Listing 5: K-Means and Hierarchical Clustering Example

Discussion

Both clustering techniques attempt to group similar data, but their mechanisms differ:

- **K-Means:** Efficient, works well for large datasets with spherical clusters.
- **Hierarchical:** Provides visual insights through dendrograms but is computationally expensive for large datasets.

The silhouette score provides a reliable measure for comparing clustering quality.

Exercises

1. Generate datasets with different shapes and densities using `make_blobs()` and `make_moons()`.
2. Apply both K-Means and Hierarchical clustering and compare silhouette scores.
3. Vary the number of clusters ($k = 2$ to $k = 6$) and observe changes in cluster separation.
4. Visualize dendrograms for multiple linkage methods (`ward`, `average`, `complete`).
5. Prepare a brief report comparing both methods.

Outcome

Students will:

- Understand and apply unsupervised clustering algorithms.
- Evaluate clustering results using silhouette scores and visualizations.
- Learn to interpret dendrograms and identify optimal cluster numbers.